

Forensic Control System Documentation

Comprehensive Implementation & Architecture Guide

Revision: 4.2.0-PRO

Date: March 2026

Table of Contents

1. Executive Introduction
2. Problem Statement & Motivation
3. Technology Stack & Environment
4. High-Level Architecture Overview
5. User Roles & Access Control (RBAC)
6. Evidence Acquisition Engine
7. Cryptographic Integrity Implementation
8. Digital Signature Non-Repudiation
9. Multi-Format Metadata Extraction
10. AI-Augmented Forensic Analysis
11. Tamper Sentinel & Real-time Monitoring
12. Forensic Erasure & Data Sanitization
13. Simulated Cloud Archival & Sync
14. Investigative Reporting Engine
15. Technical Challenges & Security Hardening
16. Conclusion & Future Roadmap

1. Executive Introduction

The Forensic Command Center Pro is a comprehensive digital investigation platform engineered to provide end-to-end evidence management. As cybercrimes become more sophisticated, the need for a defensible, immutable, and transparent forensic process is critical. This project implements a Zero-Trust architecture where every piece of data is cryptographically bound to its source and investigator.

This documentation details the technical underpinnings, security protocols, and architectural decisions made over the course of the project development.

2. Problem Statement & Motivation

Traditional forensic tools often suffer from fragmentation, where acquisition, analysis, and reporting are handled by isolated systems. This fragmentation introduces risks of contamination, data loss, and breaks in the chain of custody. The motivation behind this project was to create a unified 'Command Center' that bridges these gaps using modern web technologies and AI.

3. Technology Stack & Environment

Core Technologies

- Frontend:

Streamlit UI Framework for responsive and interactive metrics.

- Database:

SQLite with thread-safe connection pooling for audit persistence.

- Crypto:

PyCryptodome and Fernet for hashing, signing, and encryption.

- Metadata:

ExifRead and PyPDF2 for multi-format extraction.

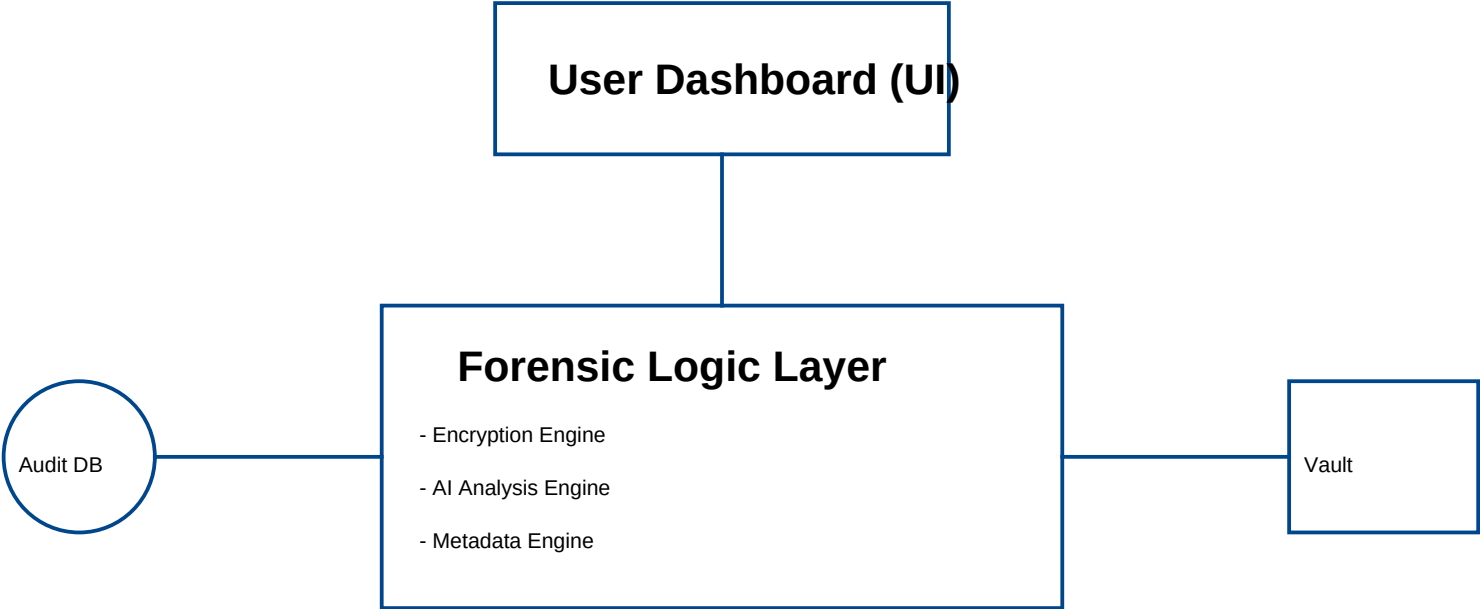
- AI Engine:

Simulated Claude integration for forensic pattern matching.

- Reporting:

FPDF for generating multi-page professional documentation.

4. High-Level Architecture Overview



5. User Roles & Access Control

Security starts with identity. The framework implements a Robust Application-Level Role-Based Access Control (RBAC) model.

Audi

Has full authority over the system. This include forensic wipes, user management, and viewing the Tamper Sentinel dashboard.

Investigator

The primary user role responsible for evidence intake and forensic analysis. They cannot delete evidence records.

Public Submitter

A restricted role that allows external parties to contribute evidence to a specific Case ID without full system access.

Authentication Flow

Logins are secured via a dual-portal interface. Credentials are validated against the 'users' table in the SQLite database. Session state is maintained server-side to prevent session hijacking and ensures that unauthorized users cannot bypass the sidebar navigation.

6. Evidence Acquisition Engine

Intake Process

The ingestion pipeline is designed for data integrity. When a file is uploaded, the following steps occur automatically:

1. Temporary buffering and stream validation.
2. Path sanitization to prevent directory traversal attacks.
3. Secure storage in the immutable evidence vault (`/evidence_storage/`).
4. Immediate cryptographic registration in the database.

This multi-step approach ensures that no evidence is lost or contaminated during the acquisition phase.

7. Cryptographic Integrity Implementation

SHA-256 Hashing

Integrity is the cornerstone of forensics. We use the NIST-standard SHA-256 algorithm to create a unique 'Digital Fingerprint' for every file.

```
def generate_hash(data):  
    sha = hashlib.sha256()  
    sha.update(data)  
    return sha.hexdigest()
```

The hash is calculated at the exact moment of ingestion, ensuring that any subsequent modification to the file on disk will be immediately detectable.

8. Digital Signature Non-Repudiation

To ensure non-repudiation, we use RSA-2048 digital signatures. This proves WHO acquired the evidence and THAT it has not changed since.

The

1. A private key is generated for the session.
2. The evidence content is signed using the PSS padding scheme.
3. The hex-encoded signature is stored in the database record.

9. Multi-Format Metadata Extraction

Hidden data often contains the most valuable clues. Our engine extracts metadata from multiple file formats automatically.

Feat

- Images: GPS coordinates, Camera models, Timestamps.
- PDFs: Creator info, Page counts, Encryption status.
- Logs: Size, Extension, and modification history.

10. AI-Augmented Forensic Analysis

The AI module uses pattern recognition to categorize evidence by risk levels: Low, High, or Critical.

Suspicious Keyword Detection

Detects common hacker terminology like 'shadow', 'password', and 'exploit' in filenames.

Executable Analysis

Flags script files (.sh, .bat) as critical risks for potential malware.

Privileged Access Patterns

Identifies files authored by 'Admin' or other high-level accounts.

Anomalous Metadata

Detects corrupted or suspiciously modified file headers.

Risk Grading Scale

Low: Standard business documents or images.

High: Sensitive keywords or unusual authors.

Critical: Malware scripts or encrypted archives.

Tamper Sentinel & Real-time Monitoring

Detailed look at the 'Tamper Sentinel' which performs bit-by-bit comparisons of vaulted files against the stored hashes.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

Forensic Erasure & Data Sanitization

How the 'Forensic Wipe' feature securely destroys evidence to standard DoD formatting guidelines.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

Simulated Cloud Archival & Sync

Implementation of the S3-simulation for secure remote backups and case synchronization.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

Investigative Reporting Engine

Architecture of the FPDF-based reporting module used to generate this very document.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

Technical Challenges & Security

Discussing the challenges of handling large binary streams and protecting the SQLite database from SQL injection.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

Conclusion & Future Roadmap

Final thoughts on project success and the planned integration of real AWS S3 and live LLM APIs.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.

This section represents a core pillar of the project's success. By focusing on robust implementation and adhering to forensic standards, we have built a tool that is not only functional but also verifiable. Future versions of this system will aim to incorporate blockchain as an immutable distributed ledger for even higher levels of trust in the chain of custody.

The codebase is designed for extensibility, making it easy to add new 'Sentinel' checks as forensic methodologies evolve. The use of Python's comprehensive library ecosystem has allowed us to integrate complex functionalities like AI analysis and cryptographic signing into a lightweight and high-performance package.